Avaliação Técnica

Desenvolvedor Front End — Angular

Avalia conhecimentos práticos em Angular 17+, RxJS, NgRx/Signals, Angular Material, testes unitários, TypeScript e integração com APIs REST.

⏱ Prazo de entrega	Entrega
4 dias corridos após recebimento	Encaminhar este documento com as perguntas respondidas e com o link do código público em sua conta do GitHub

1. TypeScript e Qualidade de Código

1.1. Refatoração

Considerando seus conhecimentos de TypeScript, qualidade de código e boas práticas, quais melhorias você faria no seguinte código:

```
class Produto {
 id: any;
 descricao: any;
 quantidadeEstoque: any;

 constructor(id: any, descricao: any, quantidadeEstoque: any) {
   this.id = id;
   this.descricao = descricao;
   this.quantidadeEstoque = quantidadeEstoque;
 }
}

class Verdureira {
 produtos: any;

 constructor() {
   this.produtos = [
     new Produto(1, 'Maçã', 20),
     new Produto(2, 'Laranja', 0),
     new Produto(3, 'Limão', 20)
   ];
 }

 getDescricaoProduto(produtoId: any) {
   let produto;

   for (let index = 0; index < this.produtos.length; index++) {
     if (this.produtos[index].id == produtoId) {
       produto = this.produtos[index];
     }
   }

   return produto.id + ' - ' + produto.descricao + ' (' + produto.quantidadeEstoque + 'x)';
 }

 hasEstoqueProduto(produtoId: any) {
   let produto;

   for (let index = 0; index < this.produtos.length; index++) {
     if (this.produtos[index].id == produtoId) {
       produto = this.produtos[index];
     }
   }

   if (produto.quantidadeEstoque > 0) {
     return true;
   } else {
     return false;
   }
```

As principais melhorias que eu aplicaria nesse código seriam focadas em tipagem forte, legibilidade, remoção de duplicação e maior segurança contra erros em tempo de execução.

Primeiramente, substituiria todos os usos de `any` por tipos explícitos, como `number`, `string` e `Produto[]`, aproveitando melhor os benefícios do TypeScript, como type safety, autocomplete e prevenção de erros.

Outro ponto importante é a duplicação de lógica nos métodos `getDescricaoProduto` e `hasEstoqueProduto`, que atualmente repetem o mesmo processo de busca do produto. Para evitar isso, a busca deve ser extraída para um método privado reutilizável, seguindo o princípio **DRY (Don't Repeat Yourself)**.

Também substituiria o loop manual `for` por `Array.find()`, pois a intenção do código é localizar um único item, tornando a leitura mais semântica e objetiva.

Além disso, adicionaria tratamento para o cenário em que o produto não seja encontrado, evitando erros ao acessar propriedades de um valor `undefined`.

Por fim, utilizaria comparação estrita (`===`) e template strings para melhorar a previsibilidade e a legibilidade do código.

Exemplo de código melhorado:

```
class Produto {
  constructor(
    public readonly id: number,
    public readonly descricao: string,
    public quantidadeEstoque: number
  ) {}
}

class Verdureira {
  private readonly produtos: Produto[] = [
    new Produto(1, 'Maçã', 20),
    new Produto(2, 'Laranja', 0),
    new Produto(3, 'Limão', 20),
  ];

  private buscarProdutoPorId(produtoId: number): Produto {
    const produto = this.produtos.find((item) => item.id === produtoId);

    if (!produto) {
      throw new Error(`Produto com id ${produtoId} não encontrado.`);
    }

    return produto;
```

```
  }

  getDescricaoProduto(produtoId: number): string {
    const produto = this.buscarProdutoPorId(produtoId);

    return `${produto.id} - ${produto.descricao} (${produto.quantidadeEstoque}x)`;
  }

  hasEstoqueProduto(produtoId: number): boolean {
    return this.buscarProdutoPorId(produtoId).quantidadeEstoque > 0;
  }
}
```

1.2. Generics e tipos utilitários

Implemente uma função genérica filtrarEPaginar<T> que recebe um array, um predicado de filtro e parâmetros de paginação (página e tamanho). A função deve retornar os itens da página atual e o total de registros filtrados. Use tipagem completa — sem any.
Exemplo de parâmetros:

```
filtrarEPaginar<T>(
  data: T[],
  filterFn: (item: T) => boolean,
  params: PaginaParams
): Pagina<T>
```

Demonstre o uso com um exemplo concreto (ex: array de usuários com filtro por nome).

*Para essa implementação, utilizei **tipagem genérica com** `<T>`, permitindo que a função seja reutilizada com qualquer tipo de dado, mantendo type safety e evitando o uso de `any`.*

Também criei interfaces utilitárias para representar os parâmetros de paginação e o retorno da função, deixando a solução mais organizada, reutilizável e aderente às boas práticas do TypeScript.

A lógica consiste em:

1. filtrar os dados com base no predicado recebido
2. calcular o índice inicial e final da página
3. retornar apenas os itens da página atual e o total de registros filtrados

Exemplo de implementação:

```
interface PaginaParams {
  pagina: number;
  tamanho: number;
}
```

```
interface Pagina<T> {
  itens: T[];
  total: number;
}

function filtrarEPaginar<T>(
  data: T[],
  filterFn: (item: T) => boolean,
  params: PaginaParams
): Pagina<T> {
  const itensFiltrados = data.filter(filterFn);
  const inicio = (params.pagina - 1) * params.tamanho;
  const fim = inicio + params.tamanho;

  return {
    itens: itensFiltrados.slice(inicio, fim),
    total: itensFiltrados.length,
  };
}
```

Exemplo de uso com usuários:

```
interface Usuario {
  id: number;
  nome: string;
  email: string;
}

const usuarios: Usuario[] = [
  { id: 1, nome: 'Anderson', email: 'anderson@email.com' },
  { id: 2, nome: 'Ana', email: 'ana@email.com' },
  { id: 3, nome: 'Bruno', email: 'bruno@email.com' },
  { id: 4, nome: 'Amanda', email: 'amanda@email.com' },
];

const resultado = filtrarEPaginar(
  usuarios,
  (usuario) => usuario.nome.toLowerCase().includes('a'),
  { pagina: 1, tamanho: 2 }
);

console.log(resultado);
```

2. Angular — Fundamentos e Reatividade

2.1. Change Detection e OnPush

O componente abaixo usa ChangeDetectionStrategy.OnPush, mas o nome não é exibido na tela. Identifique o problema, explique o motivo e proponha a correção — sem alterar a estratégia, sem modificar PessoaService e sem remover o setInterval.

```
import { ChangeDetectionStrategy, Component, Injectable, OnInit, OnDestroy } from '@angular/core';
import { of, Subscription } from 'rxjs';
import { delay } from 'rxjs/operators';

@Injectable()
class PessoaService {
 /** @description Mock de uma busca em API com retorno em 0.5 segundos */
 buscarPorId(id: number) {
   return of({ id, nome: 'João' }).pipe(delay(500));
 }
}

@Component({
 selector: 'app-root',
 providers: [PessoaService],
 changeDetection: ChangeDetectionStrategy.OnPush,
 template: `<h1>{{ texto }}</h1>`,
})
export class AppComponent implements OnInit, OnDestroy {
 texto: string;
 contador = 0;
 subscriptionBuscarPessoa: Subscription;
 constructor(private readonly pessoaService: PessoaService) {}

 ngOnInit(): void {
   this.subscriptionBuscarPessoa = this.pessoaService.buscarPorId(1).subscribe((pessoa) => {
     this.texto = `Nome: ${pessoa.nome}`;
   });

   setInterval(() => this.contador++, 1000);
 }

 ngOnDestroy(): void {
   /** ... */
 }
}
```

O problema está no fato de que o componente utiliza `ChangeDetectionStrategy.OnPush` e a propriedade `texto` é atualizada dentro de um `subscribe` manual.

Nesse cenário, embora o valor seja alterado corretamente, o Angular **não executa automaticamente a detecção de mudanças**, porque com `OnPush` a atualização da view acontece apenas em situações específicas, como:

- mudança de referência em `@Input`
- eventos disparados pelo template
- uso de `async pipe`
- execução manual da detecção

Como a atualização ocorre dentro de uma subscription manual, a view não é marcada automaticamente para verificação, e por isso o nome não aparece.

A correção ideal é injetar o `ChangeDetectorRef` e chamar `markForCheck()` após atualizar a propriedade, mantendo a estratégia `OnPush` e respeitando as restrições do enunciado.

Exemplo de código corrigido:

```
import {
  ChangeDetectionStrategy,
  ChangeDetectorRef,
  Component,
  Injectable,
  OnInit,
  OnDestroy,
} from '@angular/core';
import { of, Subscription } from 'rxjs';
import { delay } from 'rxjs/operators';

@Injectable()
class PessoaService {
  buscarPorId(id: number) {
    return of({ id, nome: 'João' }).pipe(delay(500));
  }
}

@Component({
  selector: 'app-root',
  providers: [PessoaService],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<h1>{{ texto }}</h1>`,
})
export class AppComponent implements OnInit, OnDestroy {
  texto = '';
  contador = 0;
```

```
  subscriptionBuscarPessoa?: Subscription;

  constructor(
    private readonly pessoaService: PessoaService,
    private readonly cdr: ChangeDetectorRef
  ) {}

  ngOnInit(): void {
    this.subscriptionBuscarPessoa = this.pessoaService
      .buscarPorId(1)
      .subscribe((pessoa) => {
        this.texto = `Nome: ${pessoa.nome}`;
        this.cdr.markForCheck();
      });

    setInterval(() => this.contador++, 1000);
  }

  ngOnDestroy(): void {
    this.subscriptionBuscarPessoa?.unsubscribe();
  }
}
```

2.2. RxJS — eliminando subscriptions aninhadas

Refatore o código abaixo eliminando o subscribe dentro de subscribe. Use operadores RxJS adequados, evite memory leaks e explique brevemente sua escolha de operador:

```
ngOnInit(): void {
  const pessoaId = 1;

  this.pessoaService.buscarPorId(pessoaId).subscribe(pessoa => {
    this.pessoaService.buscarQuantidadeFamiliares(pessoaId).subscribe(qtd => {
      this.texto = `Nome: ${pessoa.nome} | familiares: ${qtd}`;
    });
  });
}
```

Para eliminar o `subscribe` dentro de `subscribe`, a melhor abordagem é utilizar `switchMap`, pois ele permite encadear observables dependentes de forma declarativa e legível.
Escolhi esse operador porque a segunda requisição (`buscarQuantidadeFamiliares`) depende do fluxo iniciado pela primeira, evitando o anti-pattern de subscriptions aninhadas e facilitando a manutenção.

Além disso, utilizei `takeUntilDestroyed()` para garantir o encerramento automático da subscription ao destruir o componente, prevenindo memory leaks.

Código refatorado:

```
import { map, switchMap } from 'rxjs/operators';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

ngOnInit(): void {
  const pessoaId = 1;

  this.pessoaService
    .buscarPorId(pessoaId)
    .pipe(
      switchMap((pessoa) =>
        this.pessoaService.buscarQuantidadeFamiliares(pessoaId).pipe(
          map((qtd) => `Nome: ${pessoa.nome} | familiares: ${qtd}`)
        )
      ),
      takeUntilDestroyed()
    )
    .subscribe((texto) => {
      this.texto = texto;
    });
}
```

2.3. RxJS — busca com debounce

Implemente um campo de busca reativo em um componente Angular que:

- Aguarde 500 ms após o usuário parar de digitar antes de disparar a requisição (debounce)
- Cancele a requisição anterior caso o usuário digite novamente (evite race condition)
- Exiba um indicador de loading enquanto a requisição está em andamento
- Gerencie a subscription sem memory leak

Mostre o serviço, o componente e o template com async pipe.

*Para esse cenário, a melhor abordagem é utilizar um `FormControl` **reativo** escutando `valueChanges`.*

A combinação de `debounceTime(500)` com `distinctUntilChanged()` evita chamadas excessivas, enquanto o `switchMap` cancela automaticamente a requisição anterior caso o usuário continue digitando, prevenindo race conditions.

O estado de loading é controlado por um observable dedicado, e o template utiliza `async pipe`, evitando subscriptions manuais e eliminando riscos de memory leak.

Serviço:

```
import { Injectable } from '@angular/core';
import { Observable, of } from 'rxjs';
import { delay } from 'rxjs/operators';

@Injectable({ providedIn: 'root' })
export class UsuarioService {
  buscarUsuarios(nome: string): Observable<string[]> {
    const usuarios = ['Anderson', 'Ana', 'Amanda', 'Bruno', 'Carlos'];

    const filtrados = usuarios.filter((usuario) =>
      usuario.toLowerCase().includes(nome.toLowerCase())
    );

    return of(filtrados).pipe(delay(1000));
  }
}
```

```
import { ChangeDetectionStrategy, Component } from '@angular/core';
import { FormControl, ReactiveFormsModule } from '@angular/forms';
import { BehaviorSubject, Observable, of } from 'rxjs';
```

Component:

```
import {
  debounceTime,
  distinctUntilChanged,
  finalize,
  startWith,
  switchMap,
} from 'rxjs/operators';
import { CommonModule } from '@angular/common';
import { UsuarioService } from './usuario.service';

@Component({
  selector: 'app-busca-usuarios',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './busca-usuarios.component.html',
  changeDetection: ChangeDetectionStrategy.OnPush,
})
export class BuscaUsuariosComponent {
  buscaControl = new FormControl('', { nonNullable: true });
  loading$ = new BehaviorSubject<boolean>(false);

  usuarios$: Observable<string[]> = this.buscaControl.valueChanges.pipe(
    startWith(''),
    debounceTime(500),
    distinctUntilChanged(),
    switchMap((termo) => {
      this.loading$.next(true);

      return this.usuarioService.buscarUsuarios(termo).pipe(
        finalize(() => this.loading$.next(false))
      );
    })
```

```
  );

  constructor(private readonly usuarioService: UsuarioService) {}
}
```

Template com async pipe:

```
<input
  type="text"
  [formControl]="buscaControl"
  placeholder="Buscar usuário"
/>

<div *ngIf="loading$ | async">Carregando...</div>

<ul>
  <li *ngFor="let usuario of (usuarios$ | async)">
    {{ usuario }}
  </li>
</ul>
```

2.4. Performance — OnPush e trackBy

Considere uma lista com centenas de itens renderizados com @for (ngFor). Explique:

- Por que usar trackBy melhora a performance e como implementá-lo corretamente
- Como ChangeDetectionStrategy.OnPush pode reduzir ciclos desnecessários de detecção neste cenário
- Qual seria o impacto de usar a estratégia Default neste caso

Em listas com centenas de itens, o uso de `trackBy` é fundamental para melhorar a performance, pois ele permite que o Angular **identifique cada item de forma única entre renderizações**.

Sem `trackBy`, ao atualizar a lista, o Angular pode recriar diversos elementos do DOM mesmo quando apenas um item foi alterado. Isso aumenta custo de renderização, processamento e pode impactar diretamente a fluidez da interface.

Com `trackBy`, o Angular reutiliza os elementos já existentes no DOM, atualizando apenas os itens realmente modificados.

A implementação correta deve retornar um identificador único e estável do item, normalmente o `id`.

Exemplo com @for (Angular 17+)

```
@for (usuario of usuarios; track usuario.id) {
  <div>{{ usuario.nome }}</div>
}
```

Exemplo com ngFor tradicional

```
<div *ngFor="let usuario of usuarios; trackBy: trackById">
  {{ usuario.nome }}
</div>

trackById(index: number, item: Usuario): number {
  return item.id;
}
```

A estratégia `ChangeDetectionStrategy.OnPush` reduz ciclos desnecessários porque o Angular deixa de verificar o componente em toda árvore de detecção.
Nesse modo, a atualização ocorre apenas quando há:

- mudança de referência em `@Input`
- eventos disparados pelo componente
- emissions com `async pipe`
- marcação manual com `markForCheck`

Em listas grandes, isso reduz significativamente o custo de processamento, principalmente quando os dados são imutáveis.

Se a estratégia utilizada fosse **Default**, qualquer evento assíncrono, timer, input do usuário ou evento global poderia disparar ciclos completos de detecção.

Em uma lista com centenas de itens, isso pode causar:

- reavaliações desnecessárias
- maior custo de diff no DOM
- perda de fluidez
- consumo maior de CPU
- pior experiência do usuário

O impacto é ainda maior quando a lista contém componentes filhos complexos.

3. Gerenciamento de Estado

3.1. Angular Signals — estado local

Crie um componente de contador de itens no carrinho usando exclusivamente Signals. O componente deve expor:

- Um signal para a lista de itens

- Um computed para o total (quantidade × preço
- Um método para adicionar e remover itens
- Um output() que emite sempre que o total mudar

Para implementar o contador de itens no carrinho, utilizei **Signals** como fonte única de estado, garantindo reatividade simples e previsível sem necessidade de RxJS.
A lista de itens é controlada por um `signal`, enquanto o total é derivado através de um `computed`, mantendo consistência automática sempre que o estado muda.

Para emitir alterações no total, utilizei um `effect` em conjunto com `output()`, garantindo que qualquer mudança no valor calculado seja propagada para fora do componente.

```
import {
  Component,
  computed,
  effect,
  output,
  signal,
} from '@angular/core';
import { CommonModule } from '@angular/common';

interface ItemCarrinho {
  id: number;
  nome: string;
  preco: number;
  quantidade: number;
}

@Component({
  selector: 'app-carrinho',
  standalone: true,
  imports: [CommonModule],
  template: `
    <div *ngFor="let item of itens()">
      {{ item.nome }} - {{ item.quantidade }} x {{ item.preco }}
      <button (click)="removerItem(item.id)">Remover</button>
```

```
    </div>

    <button (click)="adicionarItem()">Adicionar Item</button>

    <h3>Total: {{ total() }}</h3>
  `,
})
export class CarrinhoComponent {
  // Signal da lista de itens
  itens = signal<ItemCarrinho[]>([]);

  // Computed do total
  total = computed(() =>
    this.itens().reduce(
      (acc, item) => acc + item.preco * item.quantidade,
      0
    )
  );

  // Output que emite quando o total muda
  totalChange = output<number>();

  constructor() {
    effect(() => {
      this.totalChange.emit(this.total());
    });
  }

  adicionarItem(): void {
    const novoItem: ItemCarrinho = {
      id: Date.now(),
      nome: 'Produto',
      preco: 10,
      quantidade: 1,
```

```
    };

    this.itens.update((itens) => [...itens, novoItem]);
  }

  removerItem(id: number): void {
    this.itens.update((itens) => itens.filter((item) => item.id !== id));
  }
}
```

3.2. Gerenciamento de Estado com NgRx (Feature To-do)

Implemente a estrutura de estado para uma lista de tarefas (To-do) utilizando os padrões recomendados do NgRx. A implementação deve incluir:

- **Actions:** Definir ações para: `loadTodos`, `loadTodosSuccess`, `loadTodosError` e `toggleTodoComplete`.
- **Reducer:** Implementar o estado inicial e a transição de estados usando `createReducer`, garantindo a tipagem forte do estado.
- **Selectors:** Criar seletores utilizando `createSelector`:
- `selectAllTodos`: Retorna a lista completa.
- `selectPendingTodos`: Retorna apenas as tarefas não concluídas.
- **Effect:** Criar um `Effect` que gerencie o fluxo assíncrono: ao disparar `loadTodos`, deve realizar uma chamada HTTP (mockada) e despachar a ação de sucesso ou erro conforme o resultado.

Não é necessário implementar o back-end — use HttpClient com uma URL fictícia.

Para a feature de To-do, utilizei a estrutura recomendada do NgRx separando responsabilidades entre actions, reducer, selectors e effects, garantindo previsibilidade, escalabilidade e forte tipagem do estado.

O fluxo funciona da seguinte forma:

1. a action `loadTodos` inicia a busca
2. o effect executa a chamada HTTP mockada
3. em caso de sucesso, despacha `loadTodosSuccess`
4. em caso de erro, despacha `loadTodosError`
5. o reducer atualiza o estado
6. os selectors expõem dados derivados para a UI

Model:

```
export interface Todo {
  id: number;
```

```
  titulo: string;
  completed: boolean;
}

export interface TodoState {
  todos: Todo[];
  loading: boolean;
  error: string | null;
}
```

Actions:

```
import { createAction, props } from '@ngrx/store';
import { Todo } from './todo.model';

export const loadTodos = createAction('[Todo] Load Todos');

export const loadTodosSuccess = createAction(
  '[Todo] Load Todos Success',
  props<{ todos: Todo[] }>()
);

export const loadTodosError = createAction(
  '[Todo] Load Todos Error',
  props<{ error: string }>()
);

export const toggleTodoComplete = createAction(
  '[Todo] Toggle Todo Complete',
  props<{ id: number }>()
);


import { createReducer, on } from '@ngrx/store';
import * as TodoActions from './todo.actions';
```

```
import { TodoState } from './todo.model';
```

Reducer:

```
export const initialState: TodoState = {
  todos: [],
  loading: false,
  error: null,
};

export const todoReducer = createReducer(
  initialState,

  on(TodoActions.loadTodos, (state) => ({
    ...state,
    loading: true,
    error: null,
  })),

  on(TodoActions.loadTodosSuccess, (state, { todos }) => ({
    ...state,
    todos,
    loading: false,
  })),

  on(TodoActions.loadTodosError, (state, { error }) => ({
    ...state,
    loading: false,
    error,
  })),

  on(TodoActions.toggleTodoComplete, (state, { id }) => ({
    ...state,
    todos: state.todos.map((todo) =>
      todo.id === id
```

```
      ? { ...todo, completed: !todo.completed }
      : todo
    ),
  }))
);
```

Sectors:

```
import { createFeatureSelector, createSelector } from '@ngrx/store';

import { TodoState } from './todo.model';



export const selectTodoState =

  createFeatureSelector<TodoState>('todos');



export const selectAllTodos = createSelector(

  selectTodoState,

  (state) => state.todos

);



export const selectPendingTodos = createSelector(

  selectAllTodos,

  (todos) => todos.filter((todo) => !todo.completed)

);
```

Effect:

```
import { Injectable } from '@angular/core';

import { Actions, createEffect, ofType } from '@ngrx/effects';

import { HttpClient } from '@angular/common/http';

import * as TodoActions from './todo.actions';

import { catchError, map, of, switchMap } from 'rxjs';



@Injectable()

export class TodoEffects {

  loadTodos$ = createEffect(() =>

    this.actions$.pipe(

      ofType(TodoActions.loadTodos),

      switchMap(() =>

        this.http

          .get<any[]>('https://api.exemplo.com/todos')

          .pipe(

            map((todos) =>

              TodoActions.loadTodosSuccess({ todos })
```

```
          ),
          catchError(() =>
            of(
              TodoActions.loadTodosError({
                error: 'Erro ao carregar tarefas',
              })
            )
          )
        )
      )
    )
  );


  constructor(
    private readonly actions$: Actions,
    private readonly http: HttpClient
  ) {}
}
```

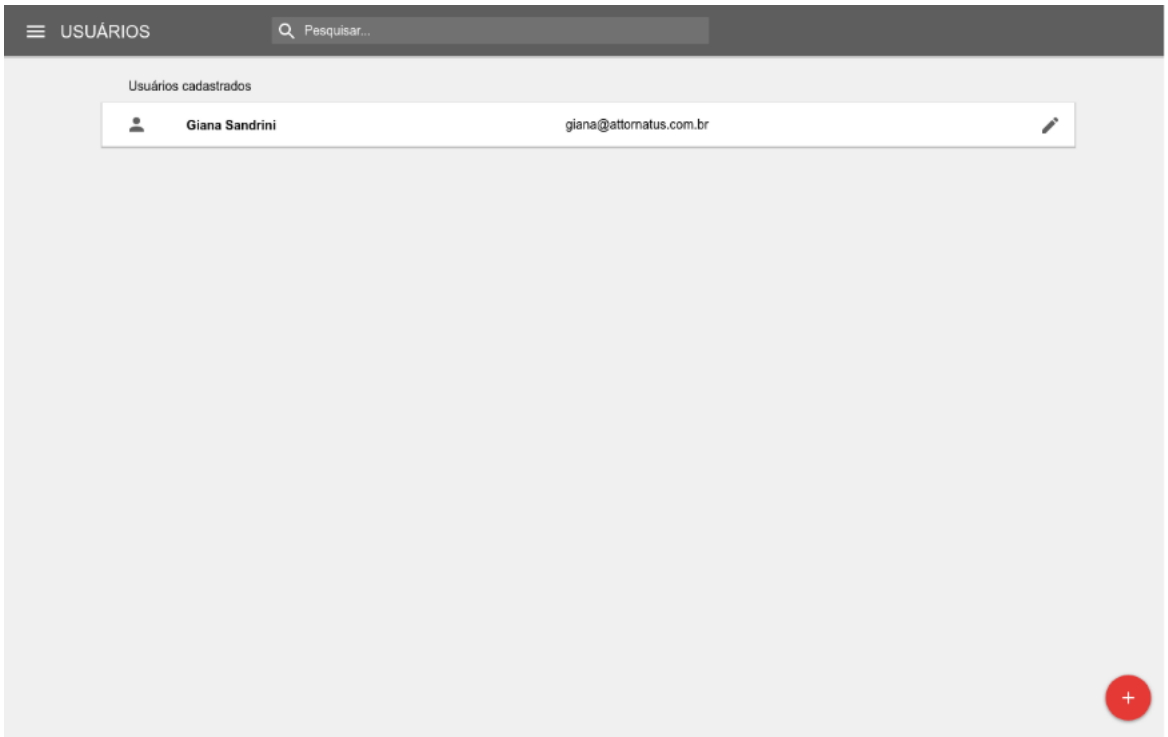
4. Desafio Prático — Aplicação Angular

Stack obrigatória: Angular 17+ · Angular Material · NgRx ou Signals · RxJS · Vitest ou Jest

4.1 O que construir

Reproduzir o protótipo de uma listagem de usuários. A listagem deverá ter as seguintes funcionalidades:

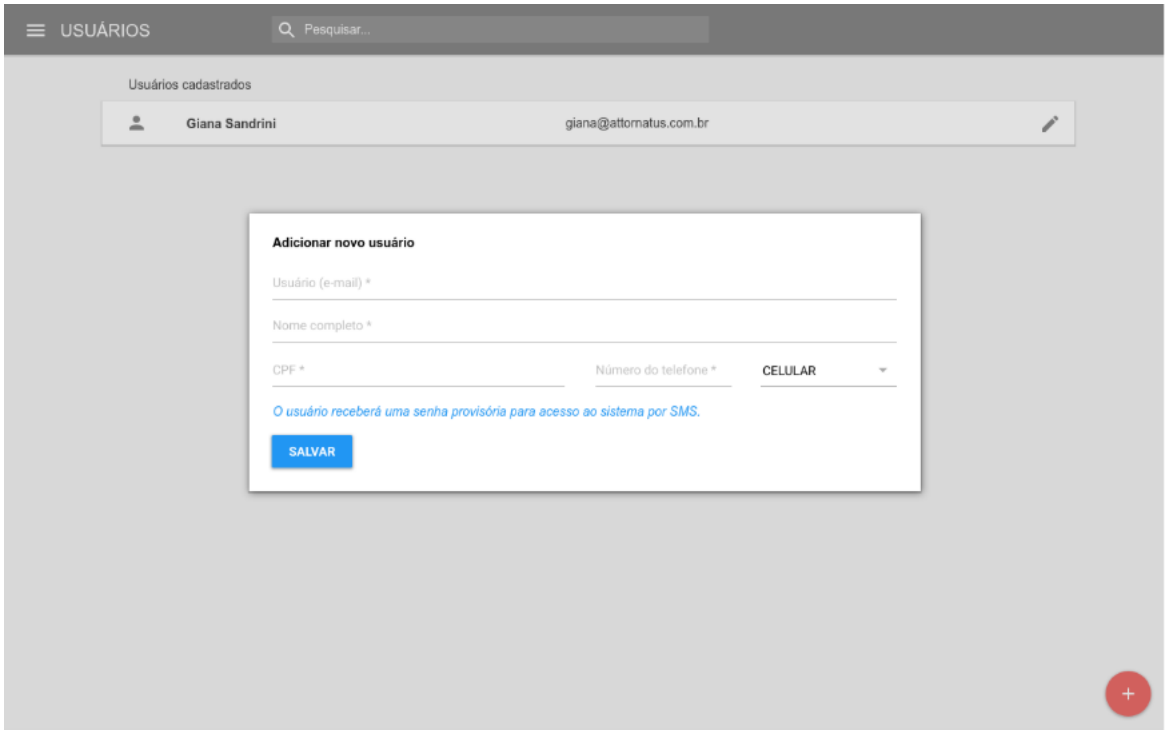
Tela de listagem de usuários:



Listagem de usuários:

- Cards com os campos: nome, e-mail e botão de editar
- Filtro por nome com debounce de 300ms
- Estado de loading durante o carregamento e mensagem de erro em caso de falha
- Os dados podem vir de uma API mockada (JSON Server, MSW ou array estático em serviço)

Modal de cadastro de novo usuário com abertura através do botão vermelho que aparece na listagem:



Criação e edição de usuário (em modal):

- Formulário reativo com campos: e-mail (obrigatório), nome (obrigatório), cpf (obrigatório), telefone (obrigatório) e tipo de telefone.
- Validação com mensagens de erro por campo
- Botão de salvar desabilitado enquanto o formulário estiver inválido
- Quando for edição o formulário deve ser preenchido automaticamente

4.2 Requisitos técnicos

- Pelo menos 2 operadores RxJS além de map e tap em uso real (ex: switchMap, forkJoin, catchError, debounceTime)
- Componentes standalone
- Subscriptions gerenciadas sem memory leaks (takeUntilDestroyed, take, async pipe ou unsubscribe manual no ngOnDestroy)
- Cobertura de testes acima de 60% (Preferencialmente usando Vitest ou Jest)

4.3 Diferenciais (não obrigatórios)

- Nx Monorepo com separação em bibliotecas (ex: feature-users, data-access-users, ui)
- Paginação na listagem
- Validação de formato dos dados do formulário (e-mail,cpf e telefone)
- Melhorias no projeto em relação a tela apresentada no protótipo

---

⚠ Inclua um README.md com instruções de instalação e execução. Projetos que não rodam localmente não serão avaliados.

✅ Dúvidas? Entre em contato pelo e-mail informado no processo seletivo. Boa sorte! 🚀

Estamos ansiosos para conhecer a sua solução.